# Cheatsheet — Lenguaje JavaScript

> **Materia:** Programación Orientada a la Web
> **Uso:** repaso rápido y consulta durante el parcial.
> **Base:** teoría de la cátedra ( `2025 Javascript_II.pdf` , 107 páginas) + estándar ECMAScript.

## Índice


1. Qué es JavaScript
2. Historia y versionado
3. Características del lenguaje
4. Cliente vs. servidor
5. Cómo incluir JavaScript en HTML
6. JavaScript vs. Java
7. Comentarios y sentencias
8. Salida: `document.write` , `alert` , `console.log` , `innerHTML`
9. Jerarquía de objetos del navegador
10. Variables: `var` , `let` , `const`
11. Tipos de datos y coerciones
12. Operadores
13. Palabras reservadas
14. Control de flujo
15. Entrada de datos: `prompt` y `confirm`
16. Arreglos
17. Funciones
18. Objetos
19. Objetos preconstruidos: `Math` y `Date`
20. Eventos
21. Strings
22. Template literals
23. DOM: qué es
24. Acceso a los nodos
25. Crear y eliminar nodos
26. Atributos, `innerHTML` y `style`
27. `addEventListener`
28. Errores típicos y checklist
29. Plantilla completa
30. Ejercicios de repaso


## 1. Qué es JavaScript

- Lenguaje **compacto de scripting basado en objetos** para desarrollar aplicaciones **Internet Cliente – Servidor**.
- Es **embebido en un documento HTML**: se coloca tanto en el `<head>` como en el `<body>` .
- Puede **reconocer y responder a eventos** del usuario **sin conexión a la red** (por ejemplo, chequeos de validación).
- Es **interpretado** por un intérprete incorporado en el navegador, que permite ejecutar código **generado dinámicamente**.
- La sintaxis es muy similar a **Java o C++**; a diferencia de ellos, el `;` final es **opcional** (aunque conviene usarlo).
- Es un lenguaje **basado en objetos** que **no implementa clases ni el mecanismo de herencia** clásico: la herencia se hace por **prototipos**.
- Usa una **jerarquía de objetos** encabezada por el navegador (**DOM**).
- Provee objetos **predefinidos** de mucha utilidad ( `Math` , `Date` , `String` , `Array` ...).
- Es la **base de AJAX** y permite incorporar otras tecnologías (XML, DHTML, ActiveX).

## 2. Historia y versionado

| Momento | Detalle |
|---|---|
| Origen | Creado por **Brendan Eich** para Netscape. |

| Momento | Detalle |
|---|---|
| Nombre original | **Mocha** → **LiveScript** → **JavaScript**. |
| Versionado | Desde JavaScript 1.0 hasta 2015. |
| Desde 2015 | Se actualiza **todos los meses de junio** con versionado **ECMAScript**. |
| Estándar actual (teoría) | **ECMAScript 2026** (1 de junio de 2026). |

> Aclaración: "JavaScript" es el nombre comercial; el **estándar** es **ECMAScript**. No confundir con Java: son lenguajes distintos.

## 3. Características del lenguaje

| Característica | Consecuencia práctica |
|---|---|
| **Interpretado** en el navegador | No requiere compilación ni máquina virtual. |
| Sintaxis tipo **Java/C++** | Fácil de leer; `;` opcional. |
| **Basado en objetos** (sin clases) | Objetos literales y herencia por **prototipos**. |
| **Jerarquía de objetos** del navegador | `window` → `document` → `forms` → `elements`. |
| **Objetos predefinidos** | `Math`, `Date`, `String`, `Array`, `Number`. |
| **Manejo de eventos** | Responde a acciones del usuario y del navegador. |
| **Débilmente tipado** | La misma variable puede cambiar de tipo en ejecución. |
| Integración | Base de **AJAX**; se combina con XML, DHTML, ActiveX. |

## 4. Cliente vs. servidor

### Del lado del cliente

- Se incluye en el documento HTML y **se descarga con él**.
- Funciones y variables dentro de `<script>...</script>`, más atributos de eventos en las etiquetas HTML.
- Lo interpretan **la mayoría de los navegadores**:
- las funciones se definen cuando **carga la página**;
- se invocan ante **eventos** (clic, cambio de valor, submit...).
- Acceso a objetos y operaciones:
- `window` (abrir ventanas, cargar URLs), `frames` (hoy obsoletos);
- elementos de un `form` (al cambiar el valor o al hacer submit);
- `links` (cambiar `href`).
- Puede crear **HTML dinámico**.

### Del lado del servidor

- Se incluye en el documento HTML "original" (`<script>` con funciones y variables).
- Lo compilan y ejecutan servidores como **Netscape LiveWire**, **Node.js**, Jaxer, EJScript, RingoJS, AppengineJS.
- Acceso a: estado del servidor, estado por usuario, requerimientos HTTP concurrentes (chequeos de usuarios y dominios), bases de datos (consultas SQL).
- Puede usarse con **cualquier navegador**.

> En la materia el foco está en el **cliente**: el código viaja al navegador y allí se ejecuta.

## 5. Cómo incluir JavaScript en HTML

### Embebido (código en la propia página)

```
<HTML>
  <HEAD>
    <SCRIPT language="JavaScript">
      alert("Hola JavaScript");
    </SCRIPT>
  </HEAD>
  <BODY>
    <P>Esto es una simple alerta JavaScript</P>
```

```
  </BODY>
</HTML>
```

> Antes se encerraba el código entre comentarios `<!-- ... -->` para que los navegadores viejos no lo mostraran. Hoy **no es necesario**.

### Enlazado (archivo externo `.js`)

```
<HTML>
  <HEAD>
    <SCRIPT language="JavaScript" SRC="codigo.js"></SCRIPT>
  </HEAD>
  <BODY>
  </BODY>
</HTML>
```

### ¿En el `HEAD` o en el `BODY`?

**Colocar los scripts al final del `<body>` mejora la velocidad de visualización**, porque la interpretación de los scripts **ralentiza** el renderizado de la página.

```
<HTML>
  <HEAD></HEAD>
  <BODY>
    <!-- contenido visible primero -->
    <SCRIPT language="JavaScript" SRC="codigo.js"></SCRIPT>
  </BODY>
</HTML>
```

| Forma | Ventaja | Desventaja |
|---|---|---|
| Embebido | Todo en un archivo; útil para pruebas. | Se mezcla con el HTML, no se reutiliza. |
| Enlazado | Reutilizable y cacheable; HTML limpio. | Una petición más al servidor. |
| En `head` | Disponible antes de dibujar el body. | Bloquea el renderizado. |
| Al final del `body` | La página se muestra antes. | El DOM ya está listo al ejecutarse. |

---

## 6. JavaScript vs. Java

| JAVA | JAVASCRIPT |
|---|---|
| Requiere **compilación**. | **No** requiere compilación: lo interpreta el navegador del lado del cliente. |
| Lenguaje **pesado, potente y robusto**. | Lenguaje **ligero, ágil y poco robusto**. |
| **Orientado a objetos** completamente. | No se clasifica bajo un paradigma concreto. |
| Requiere **JDK y máquina virtual**. | Solo un **navegador** y un editor de texto. |
| **Fuertemente tipado**. | **Débilmente tipado**: las variables cambian de tipo en ejecución. |
| Clases e instancias son entidades **distintas**. | **Todos los objetos son instancias**. |
| Clase con definición y **constructor**. | Objetos con **funciones constructoras**. |
| Jerarquía por **subclases**. | Jerarquía por **prototipos**. |
| Herencia por **cadena de clases**. | Herencia por **cadena de prototipos**. |
| La clase define **todas** las propiedades de sus instancias. | El constructor/prototipo define un conjunto **inicial**; se pueden **agregar o quitar propiedades** en ejecución. |

> Moraleja para el parcial: **Java ≠ JavaScript**. Solo comparten parte de la sintaxis y el nombre.

---

## 7. Comentarios y sentencias

```
// Esta es una línea de comentario

/* Este es un
   comentario multilínea */

var x = 10;   // sentencias separadas por ;
```

- Los comentarios son **ignorados por el intérprete**.
- El `;` es **opcional**, pero **clarifica** la separación de instrucciones.
- JavaScript **distingue mayúsculas y minúsculas** (`nombre`, `Nombre` y `NOMBRE` son distintas).

## 8. Salida: `document.write`, `alert`, `console.log`, `innerHTML`

| Forma | Para qué | Ejemplo |
|---|---|---|
| `document.write()` | Escribe en el documento **mientras se carga**. | `document.write("Hola<br>");` |
| `alert()` | Ventana modal con un mensaje y botón **OK**. | `alert("Hola JavaScript");` |
| `console.log()` | Salida por consola, para **depuración**. | `console.log(5 + 6);` |
| `innerHTML / innerText` | Escribe **dentro de un elemento**. | `document.getElementById("demo").innerHTML = "Hola";` |

```
<SCRIPT>
  document.write("Hola Mundo" + "<BR>");
</SCRIPT>
```

- El texto puede ir con **comillas simples o dobles**.
- Se pueden **embeber etiquetas HTML** en el texto: el navegador las interpreta.
- Para que las etiquetas se **vean** en pantalla hay que usar **caracteres de escape** `<` y `>`:

```
document.write("<B>Hola JavaScript</B>");              // se ve en negrita
document.write("&lt;B&gt;Hola JavaScript&lt;/B&gt;");  // se ve la etiqueta
```

> `document` es el objeto documento y `write` es un **método** de ese objeto: `objeto.metodo()`.

**`document.write` vs. `innerHTML`**

| | `document.write` | `innerHTML` |
|---|---|---|
| Dónde escribe | En la página actual, **borrando** lo que había. | **Dentro** de un elemento existente. |
| Cuándo usarlo | Al generar la página mientras carga. | Para actualizar una parte de la página. |
| Uso moderno | Poco recomendado. | Muy usado (y base de librerías como jQuery). |

## 9. Jerarquía de objetos del navegador

```
window
 ├── parent, frames, self, top, location, history
 └── document
      ├── forms
      │    └── elements
      │         (text, textarea, checkbox, radio, password,
      │          select, button, submit, reset)
      ├── links
      └── anchors
```

- El **DOM** se puede **acceder y manipular** con JavaScript.
- Cada objeto tiene **propiedades y métodos**.
- La jerarquía **refleja la estructura** de la página HTML: los descendientes de un objeto son **propiedades** de ese objeto.

```
document.title = "un simple documento";
document.myform.length = 5;
document.forms[0].buttons[0].value = "Aceptar";
```

## 10. Variables: `var`, `let`, `const`

### Las 4 formas de definir una variable

| Forma | Ejemplo | Comentario |
|---|---|---|
| `var` | `var pepe;` | Ámbito **global** o de **función**. |
| `const` | `const PI = 3.14159;` | Valor **fijo**; no se puede redefinir. |
| `let` | `let x = 5;` | Valor **variable**; no se puede **redeclarar**. |

| Forma | Ejemplo | Comentario |
|---|---|---|
| Sin declarar | `x = 2;` | **Mala práctica**: crea una global. |

> `let` y `const` se agregaron en **2015** y **no funcionan** en navegadores que no contemplen el estándar.

## Declaración y asignación

```
var pepe;                  // declaración
pepe = "Ser o no ser";     // asignación

var primerNombre = "";     // conviene inicializar con el tipo esperado
var anio = 0;
var vive = true;
```

- La asignación es **siempre de derecha a izquierda**.
- Las variables **pueden reasignarse** y **cambiar de tipo**:

```
var dato = 0;
dato = "hola mundo";   // válido: JS no es tipado
```

### `const` no se redefine

```
const PI = 3.141592653589793;

PI = 3.14;      // ❌ no válido
PI = PI * 3;    // ❌ no válido
```

### `let` no se redeclara, pero sí se reasigna

```
let x = 4;
let x = 54;   // ❌ ERROR: ya fue declarada con let

let y = 4;
y = 54;       // ✅ válido: es reasignación, no redeclaración
```

> **Redefinir** (volver a declarar) ≠ **reasignar** (cambiar el valor).

## Ámbito por bloque

- `var` (o una variable sin declarar) tiene ámbito **global** o de la **función**.
- `let` y `const` pueden limitarse a un **bloque** `{ }`.

```
let x = 5;

{
  let x = 10;    // otra x, solo dentro del bloque
  console.log(x); // 10
}

console.log(x);   // 5  (al salir del bloque se conserva la de afuera)
```

## Reglas de nombres

- Distinguen **mayúsculas/minúsculas**: `PEPE`, `Pepe` y `pepe` son distintas.
- Deben empezar con **letra** o **guion bajo** `_`; luego letras, números o `_`.
- No pueden empezar con número ni usar palabras reservadas.

## Caracteres especiales en cadenas

| Escape | Significado |
|---|---|
| `\n` | Salto de línea |
| `\t` | Tabulación |
| `\r` | Retorno de carro |
| `\b` | Retroceso |
| `\f` | Salto de página |
| `\"` | Comilla doble |

## 11. Tipos de datos y coerciones

### Tres tipos de datos

| Tipo | Ejemplos |
|---|---|
| **String** (cadena) | `"Hola JavaScript"`, `'Hola JavaScript'`, `""` |
| **Number** (número) | `1`, `5`, `3.14`, `65000` |
| **Boolean** (booleano) | `true`, `false` |

> Además existen `null`, `undefined`, `object`, `function` y (moderno) `symbol` / `bigint`.

### No es tipado: conversión automática

```
var r = 2;
r = r + 5;          // 7        (suma)

var r = "2";
r = r + 5;          // "25"     (concatena string + número)

var r = "2";
r = r * 5;          // 10       (multiplica)

var r = "2";
r = r - 5 + r;      // "-32"    (resta 2-5 = -3, luego concatena)

var r = "ra" + 16;  // "ra16"
var r = 2 * 3 + "ra"; // "6ra"
```

### Reglas prácticas de la coerción

| Operación | Resultado |
|---|---|
| `+` con un string | **Concatena** (convierte el otro operando a string). |
| `-`, `*`, `/`, `%` | **Convierte a número** y opera. |
| `"5" == 5` | `true` (compara valor, convirtiendo). |
| `"5" === 5` | `false` (compara valor **y tipo**). |

> Como JS no es tipado, **hay que respetar el contenido** de las variables. Un tipo inesperado produce resultados raros.

`prompt()` **siempre devuelve string**: hay que convertir con `parseInt()`, `parseFloat()` o `Number()`.

---

## 12. Operadores

### String

```
var texto = "Hola" + " " + "mundo";   // concatenación con +
```

### Numéricos

| Operador | Significado |
|---|---|
| `+` | Suma |
| `-` | Resta |
| `*` | Multiplicación |
| `/` | División |
| `%` | Módulo (resto de la división) |
| `++` | Incremento en 1 |
| `--` | Decremento en 1 |

```
y = x++;   // usa x y luego incrementa
y = ++x;   // incrementa y luego usa x
```

### Igualdad y comparación

| Operador | Significado |
|---|---|
| `==` | Igual a (compara valor) |
| `!=` | Distinto |
| `>` / `>=` | Mayor / mayor o igual |
| `<` / `<=` | Menor / menor o igual |
| `===` / `!==` | Igualdad / desigualdad **estricta** (valor y tipo) |

### Lógicos

| Operador | Significado |
|---|---|
| `&&` | AND lógico |
| `\|\|` | OR lógico |
| `!` | NOT |

### Ternario

```
var mensaje = (edad >= 18) ? "Mayor" : "Menor";
```

### Precedencia (resumen)

```
!  ++  --
*  /  %
+  -
<  <=  >  >=
==  !=  ===  !==
&&
||
=  +=  -=  ...
```

> Ante la duda, **paréntesis**.

---

## 13. Palabras reservadas

| Palabra | Función |
|---|---|
| `break` | Termina (corta) el bucle actual y pasa a la sentencia siguiente. |
| `comment` | Anotaciones del autor; el intérprete las ignora. |
| `continue` | Termina la iteración actual y continúa con la próxima. |

Otras de uso habitual: `var`, `let`, `const`, `function`, `return`, `if`, `else`, `for`, `while`, `switch`, `case`, `default`, `new`, `this`, `true`, `false`, `null`, `typeof`, `delete`, `in`, `do`.

---

## 14. Control de flujo

### `if` / `else if` / `else`

```
var color = "azul";

if (color == "rojo") {
  document.write("el color es rojo");
  document.write("<HR>");
} else if (color == "azul") {
  document.write("el color es azul");
} else {
  document.write("el color no es ni rojo ni azul");
}
```

### `switch`

```
switch (color) {
  case "rojo":
    document.write("Rojo");
```

```
    break;
  case "azul":
    document.write("Azul");
    break;
  default:
    document.write("Otro color");
}
```

**`for`**

```
for (var i = 0; i <= 1000; i = i + 100) {
  document.write(i + "<BR>");
}
```

**`for...in`**

Itera una variable sobre **todas las propiedades de un objeto** (o los índices de un arreglo).

```
<script type="text/javascript">
  var x;
  var misautos = new Array();
  misautos[0] = "Ferrari";
  misautos[1] = "Volvo";
  misautos[2] = "BMW";

  for (x in misautos) {
    document.write(misautos[x] + "<br />");
  }
</script>
```

**`while`**

```
var i = 0;
while (i <= 1000) {
  document.write(i + "<BR>");
  i = i + 100;
}
```

**`do...while`**

```
var i = 0;
do {
  document.write(i + "<BR>");
  i++;
} while (i <= 10);
```

| Bucle | Cuándo usarlo |
|---|---|
| `for` | Cantidad de iteraciones conocida. |
| `for...in` | Recorrer propiedades de un objeto o índices de un arreglo. |
| `while` | Mientras se cumpla una condición (puede no ejecutarse). |
| `do...while` | Al menos una ejecución garantizada. |

## 15. Entrada de datos: `prompt` y `confirm`

| Función | Comportamiento | Devuelve |
|---|---|---|
| `alert(mensaje)` | Muestra un mensaje; se cierra con **OK**. | Nada |
| `confirm(mensaje)` | Dos botones: **Aceptar** / **Cancelar**. | `true` / `false` |
| `prompt(str1, str2)` | Pide un dato. `str1` = pregunta, `str2` = texto por defecto. | **String** (o `null`) |
| `console.log(x)` | Escribe en la consola (depuración). | Nada |

```
if (confirm("¿Quiere imprimir texto grande en rojo?") == true) {
  document.write("<DIV STYLE='color:red;font-size:36pt;'>texto grande en rojo</DIV>");
} else {
  document.write("texto pequeño en negro");
}
```

```
var suNombre = prompt(
  "Por favor escriba su nombre",
  "(sobreescriba este texto con su nombre)"
);
document.write("Su nombre es: " + suNombre);
```

> `prompt` devuelve **texto**. Para operar: `var edad = parseInt(prompt("Edad", ""));`

---

## 16. Arreglos

### Creación

```
var nombreArray = new Array();     // arreglo vacío
var nombreArray = new Array(10);   // 10 posiciones para almacenar
```

### Carga y acceso

```
var gatos = new Array(4);
gatos[0] = "Tom";
gatos[1] = "Lucas";
gatos[2] = "Adam";
gatos[3] = "Luis";

// O directamente:
var gatos = new Array("Tom", "Lucas", "Adam", "Luis");

// Sintaxis moderna:
var gatos = ["Tom", "Lucas", "Adam", "Luis"];
```

### `length` y extensión

```
var n = gatos.length;   // 4

gatos[9] = "Nuevo";     // el arreglo se extiende solo
gatos.length = 2;       // también se puede recortar
```

> Los arreglos son **útiles para crear, ordenar y manipular listas** usando bucles.

### Métodos y propiedades

| Método | Para qué |
|---|---|
| `length` | Cantidad de elementos (propiedad). |
| `sort()` | Ordena alfabéticamente (modifica el arreglo). |
| `reverse()` | Invierte el orden. |
| `pop()` | Quita el **último** elemento. |
| `push()` | Agrega al **final**. |
| `join(sep)` | Une los elementos en un string. |
| `toString()` | Convierte el arreglo a string separado por comas. |

```
var frutas = ["pera", "manzana", "kiwi"];

frutas.sort();           // ["kiwi", "manzana", "pera"]
frutas.reverse();        // ["pera", "manzana", "kiwi"]
frutas.push("uva");      // agrega al final
var ultima = frutas.pop(); // "uva"
var texto = frutas.join(" - ");  // "pera - manzana - kiwi"
```

---

## 17. Funciones

- Hacen cualquier cosa: desde imprimir un mensaje hasta calcular y **retornar un valor**.
- Pueden **recibir datos**, manipularlos y **devolver** un resultado.

### Declaración y llamada

```
function nombreFuncion() {
  // código de la función
}

var x = nombreFuncion();   // llamada
```

### Ejemplo sin parámetros

```
function warning() {
  document.write("<DIV STYLE='color:red;text-align:center;font-size:32pt;'>Warning!</DIV>");
  document.write("<BR><DIV STYLE='color:green;text-align:center;font-size:24pt;'>Ud. ha sido avisado!</DIV>");
}

warning();
```

### Ejemplo con parámetros y `return`

```
function multiplicar(a, b) {
  var producto = 0;
  producto = a * b;
  return producto;
}

var x = multiplicar(3, 5);
document.write(x);   // 15
```

| Concepto | Detalle |
|---|---|
| Parámetros | Se declaran entre paréntesis y sin tipo. |
| `return` | Devuelve un valor y **corta** la ejecución de la función. |
| Sin `return` | La función devuelve `undefined`. |
| Orden | La función debe estar **definida antes** de ejecutarse (o usar *hoisting* de declaraciones). |

---

## 18. Objetos

### Objetos personalizados con `new Object`

```
var remera = new Object();
remera.color = "roja";
remera.size = 8;
remera.type = "algodon";

document.write(remera.color + "<BR>");
document.write(remera.size + "<BR>");
document.write(remera.type + "<BR>");
```

### Qué es un objeto

- Son **variables que pueden almacenar tanto valores como funciones**.
- Son **colecciones de propiedades**, y las propiedades se pueden **modificar, añadir y eliminar**.
- Los objetos **preconstruidos** ya traen funciones para manejarlos (por ejemplo `Date` y `Math`).

### Tres formas de declarar un objeto

```
// 1. Objeto literal
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
};

// 2. Crear vacío y agregar propiedades
const person = {};
person.firstName = "John";
person.lastName = "Doe";
person.age = 50;
person.eyeColor = "blue";

// 3. Con new Object y un literal
```

```
const person = new Object({
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
});
```

```
const car = {
  type: "Fiat",
  model: "500",
  color: "white"
};
```

### Métodos y `this`

```
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  fullName: function () {
    return this.firstName + " " + this.lastName;
  }
};

person.fullName();   // "John Doe"
```

```
const persona = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  getId: function () {
    return this.id;
  }
};

let number = persona.getId();   // 5566
```

### Agregar un método después

```
person.name = function () {
  return this.firstName + " " + this.lastName;
};

// Reutilizando métodos de string
person.name = function () {
  return (this.firstName + " " + this.lastName).toUpperCase();
};
```

| Concepto | Detalle |
|---|---|
| **Propiedad** | Par `clave: valor` (o `objeto.clave = valor`). |
| **Método** | Propiedad cuyo valor es una **función**. |
| `this` | Dentro de un método, referencia **al objeto** que lo contiene. |
| Acceso | `objeto.propiedad` o `objeto["propiedad"]`. |

---

## 19. Objetos preconstruidos: `Math` y `Date`

`Math`

**Constantes:** `E`, `LN10`, `LN2`, `LOG10E`, `LOG2E`, `PI`, `SQRT1_2`, `SQRT2`.

**Métodos:** `abs`, `acos`, `asin`, `atan`, `atan2`, `ceil`, `cos`, `exp`, `floor`, `log`, `max`, `min`, `pow`, `random`, `round`, `sin`, `sqrt`, `tan`.

```
Math.PI;                // 3.141592653589793
Math.max(3, 7, 2);      // 7
Math.round(4.6);        // 5
Math.floor(4.9);        // 4
Math.ceil(4.1);         // 5
Math.sqrt(16);          // 4
Math.pow(2, 8);         // 256
Math.random();          // 0 <= x < 1
parseInt(Math.random() * 10);   // 0 a 9
```

`Date`

```
var miDia = new Date();
document.write(miDia + "<BR>");

var x = miDia.toGMTString();
document.write(x + "<BR>");
```

| Método | Devuelve |
|---|---|
| `getFullYear()` | Año (4 cifras). |
| `getMonth()` | Mes (**0–11**). |
| `getDate()` | Día del mes (1–31). |
| `getDay()` | Día de la semana (**0 = domingo**). |
| `getHours()` / `getMinutes()` / `getSeconds()` | Hora. |
| `toGMTString()` | Fecha en formato GMT. |

> `getMonth()` arranca en **0**: hay que sumarle 1 para mostrar el mes real.

---

## 20. Eventos

### Niveles de eventos

1. A nivel del **documento HTML**.
2. A nivel de un **formulario**.
3. A nivel de un **elemento de un formulario**.

El evento lo gestiona una sección de código JavaScript: el **gestor (manejador) de eventos**.

### Declaración de gestores

Igual que los atributos HTML:

```
<BODY onLoad="cargarfuncion()" onUnload="descargarfuncion()">

<FORM name="nombre_del_formulario" onSubmit="funcion_o_sentencia">

<INPUT type="button" name="mycheck" value="HA!"
       onClick="alert('Te he dicho que no me aprietes')">
```

### Ejemplo básico

```
<HTML>
  <HEAD>
    <SCRIPT>
      function funcionClick() {
        alert("Ud ha cliqueado");
      }
    </SCRIPT>
  </HEAD>
  <BODY>
    <H2 onclick="funcionClick()">Cliqueame</H2>
  </BODY>
</HTML>
```

### Validar un link antes de navegar

```
<a href="mi_pagina.html"
   onclick="return confirm('Ud. Quiere ir a mi_pagina.html?');">
  ir a mi pagina
</a>

<!-- O con una función que devuelva true/false -->
<a href="mi_pagina.html" onclick="return confirmar();">ir a mi pagina</a>
```

> El `return` es clave: devuelve al comportamiento de la etiqueta el `true` / `false`. Si devuelve `false`, **no navega**.

## Validar un formulario

```
<script>
function validateForm() {
  var x = document.forms["myForm"]["fname"].value;

  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
</script>

<form name="myForm" action="/action_page.php"
      onsubmit="return validateForm()" method="post">
  Name: <input type="text" name="fname">
  <input type="submit" value="Submit">
</form>
```

> Hoy la mayoría de estas validaciones se hacen con **HTML5** (`required`), que evita todo el código anterior.

## Tabla de eventos y manejadores

| Evento | Manejador | Cuándo se ejecuta |
|---|---|---|
| Abort | `onAbort` | El usuario aborta la carga de una imagen. |
| Blur | `onBlur` | Un elemento de formulario, ventana o frame **pierde el foco**. |
| Change | `onChange` | Select, text o textarea pierde el foco y **el valor cambió**. |
| Click | `onClick` | Se hace clic sobre un objeto. |
| DblClick | `onDblClick` | Doble clic sobre un elemento o enlace. |
| Mouse Over | `onMouseOver` | El puntero pasa **por encima** de un objeto o área. |
| Mouse Out | `onMouseOut` | El puntero **sale** del objeto. |
| Load | `onLoad` | Termina la carga del documento. |
| Unload | `onUnload` | Se descarga el documento. |
| Submit | `onSubmit` | Se envía un formulario. |
| Reset | `onReset` | Se resetea un formulario. |
| Focus | `onFocus` | El elemento **recibe** el foco. |
| KeyDown / KeyPress / KeyUp | `onKeyDown` / `onKeyPress` / `onKeyUp` | Se presiona / tipea / suelta una tecla. |
| MouseDown / MouseUp | `onMouseDown` / `onMouseUp` | Se presiona / suelta un botón del mouse. |
| MouseMove | `onMouseMove` | El mouse se mueve. |
| Select | `onSelect` | Se selecciona texto. |
| Resize | `onResize` | Se redimensiona la ventana. |
| Error | `onError` | Ocurre un error al cargar una imagen o script. |
| Move | `onMove` | Se mueve la ventana. |
| DragDrop | `onDragDrop` | Se arrastra y suelta un objeto. |

## `this` en los eventos

`this` hace referencia **al objeto que produjo el evento**, y permite acceder a sus atributos (por ejemplo `value`).

```
<input type="text" onChange="checaCampo(this)">
```

## Varias instrucciones en un manejador

```
<input type="text" onChange="
  if ( parseInt(this.value) <= 5 ) {
    alert('Ponga un número mayor que 5.');
  }
">
```

### `onLoad` y `onUnload`

```
<body onLoad="alert('Bienvenido!');"
      onUnload="alert('Adios!');">
  <h1>Página efusiva</h1>
</body>
```

```
// Abrir una ventana al descargar la página
onUnload="window.open('publicidad.html');"
```

> Algunos navegadores **bloquean** las ventanas emergentes según la configuración de confianza del cliente.

### Cambiar una imagen con `onMouseOver` / `onMouseOut`

```
<a href="#"
   onMouseOver="document.the_image.src='stuff/js_on.gif';"
   onMouseOut="document.the_image.src='stuff/js_off.gif';">
  <img src="stuff/js_des.gif" name="the_image" border="0">
</a>
```

> La imagen cambia al pasar por encima y **vuelve** al salir.

---

## 21. Strings

Un **string** es cero o más caracteres entre comillas (dobles o simples).

```
var cad  = "esto es una cadena";
var cad2 = "";
var cad3 = 'hola';

let largo = cad.length;   // 16
```

### Métodos de extracción y búsqueda

| Método | Para qué |
|---|---|
| `length` | Longitud (propiedad). |
| `slice(inicio, fin)` | Extrae una parte (acepta negativos). |
| `substring(inicio, fin)` | Extrae una parte. |
| `substr(inicio, largo)` | Extrae desde `inicio` la cantidad indicada. |
| `charAt(pos)` | Carácter en la posición. |
| `charCodeAt(pos)` | Código Unicode del carácter. |
| `indexOf(texto)` | Posición de la **primera** aparición (-1 si no está). |
| `lastIndexOf(texto)` | Posición de la **última** aparición. |
| `search(texto)` | Busca y devuelve la posición. |
| `match(regex)` | Busca coincidencias con una expresión regular. |
| `matchAll(regex)` | Todas las coincidencias. |
| `includes(texto)` | ¿Contiene el texto? ( `true` / `false` ). |
| `startsWith(texto)` / `endsWith(texto)` | ¿Empieza / termina con? |

### Métodos de transformación

| Método | Para qué |
|---|---|
| `replace(a, b)` | Reemplaza la **primera** aparición. |
| `replaceAll(a, b)` | Reemplaza **todas**. |
| `toUpperCase()` / `toLowerCase()` | Mayúsculas / minúsculas. |
| `concat(texto)` | Concatena (equivalente a `+` ). |
| `trim()` | Quita espacios al inicio y al final. |
| `trimStart()` / `trimEnd()` | Quita solo al inicio / solo al final. |

| Método | Para qué |
|---|---|
| `padStart(largo, relleno)` / `padEnd(...)` | Rellena al inicio / al final. |
| `split(separador)` | Divide el string en un **arreglo**. |

```
let texto = "  Hola Mundo  ";

texto.trim();               // "Hola Mundo"
texto.trim().toUpperCase(); // "HOLA MUNDO"
"Hola Mundo".includes("Mundo");   // true
"Hola Mundo".indexOf("o");        // 1
"Hola Mundo".slice(0, 4);         // "Hola"
"a,b,c".split(",");               // ["a", "b", "c"]
"5".padStart(3, "0");             // "005"
```

## 22. Template literals

- Usan **comillas invertidas** ( ` ).
- Las comillas simples o dobles **dentro** se muestran sin escapar.
- Permiten **multilínea** (con comillas comunes no se puede).
- Permiten **interpolar** variables y expresiones con `${...}`.

```
var nombre = `el nombre es "pep" `;

var texto = `esto es
un texto`;
```

```
let nombre = "Juan";
let apellido = "Perez";
let texto = `mi nombre es ${nombre} ${apellido}`;

let price = 10;
let VAT = 0.25;
let total = `Total: ${(price * (1 + VAT)).toFixed(2)}`;   // "Total: 12.50"
```

> Los *template literals* son la forma moderna de **armar cadenas con variables**; reemplazan a la concatenación con `+`.

## 23. DOM: qué es

**DOM** = *Document Object Model* ("Modelo de Objetos del Documento" / "Modelo en Objetos para la Representación de Documentos").

- Es una **API** que proporciona un conjunto **estándar de objetos** para representar documentos **HTML y XML**.
- Define un modelo estándar de **cómo se combinan** esos objetos y una **interfaz estándar** para accederlos y manipularlos.
- A través del DOM los programas pueden **acceder y modificar el contenido, la estructura y el estilo** de los documentos HTML y XML.
- El responsable del DOM es el **W3C** (*World Wide Web Consortium*).
- Es la interfaz que usa **ECMAScript (JavaScript)** para trabajar con documentos estructurados.

### Instanciación: del HTML a los objetos

```
<HTML>
  <HEAD>
    <TITLE>Ejemplo sencillo de página HTML</TITLE>
  </HEAD>
  <BODY>
    <A name="principio">Este es el principio de la página</A>   <!-- ancla -->
    <HR>
    <FORM method="POST">
      <P>Introduzca su nombre: <INPUT type="text" name="me" size="70"></P>
      <INPUT type="reset" value="Borrar Datos">
      <INPUT type="submit" value="OK">
    </FORM>
    <HR>
    Clica aquí para ir al
    <A href="#principio">principio</A> de la página              <!-- link -->
  </BODY>
</HTML>
```

```
document.title                    // "Ejemplo sencillo de página HTML"
document.anchors[0].name          // "principio"
document.forms[0].method          // "post"
document.forms[0].elements[1].value
document.links[0].href
```

## Jerarquía de objetos del navegador

```
window
  + history
  + location
  + document          <BODY> ... </BODY>
     - anchor          <A NAME="..."> ... </A>
     - applet          <APPLET> ... </APPLET>
     - area            <MAP> ... </MAP>
     - form            <FORM> ... </FORM>
        + button       <INPUT TYPE="button">
        + checkbox     <INPUT TYPE="checkbox">
        + fileUpload   <INPUT TYPE="file">
        + hidden       <INPUT TYPE="hidden">
        + password     <INPUT TYPE="password">
        + radio        <INPUT TYPE="radio">
        + reset        <INPUT TYPE="reset">
        + select       <SELECT> ... </SELECT>
           - options   <OPTION> ... </OPTION>
        + submit       <INPUT TYPE="submit">
        + text         <INPUT TYPE="text">
        + textarea     <TEXTAREA> ... </TEXTAREA>
     - image           <IMG SRC="...">
     - link            <A HREF="..."> ... </A>
     - plugin          <EMBED SRC="...">
  + frame              <FRAME>
```

## Objeto `window`: propiedades

| Propiedad | Contenido |
|---|---|
| `closed` | `true` si la ventana está cerrada. |
| `defaultStatus` | Texto por defecto de la barra de estado (hoy deshabilitada). |
| `frames` | Array con los frames de la ventana. |
| `history` | Array con las URLs visitadas. |
| `length` | Cantidad de frames de la ventana. |
| `location` | URL de la barra de dirección. |
| `name` | Nombre de la ventana o frame. |
| `opener` | Referencia a la ventana que la abrió con `open()`. |
| `parent` | Ventana que contiene el frameset. |
| `self` | Nombre alternativo de la ventana actual. |
| `status` | Mensaje de la barra de estado. |
| `top` | Ventana de nivel superior. |
| `window` | Igual a `self`. |

## Objeto `window`: métodos

| Método | Qué hace |
|---|---|
| `alert(mensaje)` | Cuadro de diálogo con el mensaje. |
| `confirm(mensaje)` | Aceptar / Cancelar → `true` / `false`. |
| `prompt(mensaje, porDefecto)` | Cuadro con caja de texto. |
| `blur()` / `focus()` | Quita / captura el foco de la ventana. |
| `close()` | Cierra la ventana actual. |
| `open(URL, nombre, características)` | Abre una ventana nueva. |
| `moveBy(x, y)` / `moveTo(x, y)` | Mueve la ventana. |
| `scroll(x, y)` / `scrollBy(x, y)` / `scrollTo(x, y)` | Desplaza la ventana. |

| Método | Qué hace |
|---|---|
| `setInterval(expr, ms)` | Ejecuta la expresión **cada** `ms` ms. |
| `setTimeout(expr, ms)` | Ejecuta la expresión **una vez**, tras `ms` ms. |
| `clearInterval(id)` / `clearTimeout(id)` | Cancela el temporizador. |

### `window.open()` : características

```
window.open("pagina.html", "miVentana",
  "width=400,height=300,toolbar=no,scrollbars=yes,resizable=yes");
```

| Característica | Valores | Significado |
|---|---|---|
| `toolbar` | `yes/no/1/0` | Barra de herramientas. |
| `location` | `yes/no/1/0` | Campo de localización. |
| `directories` | `yes/no/1/0` | Botones de dirección. |
| `status` | `yes/no/1/0` | Barra de estado. |
| `menubar` | `yes/no/1/0` | Barra de menús. |
| `scrollbars` | `yes/no/1/0` | Barras de desplazamiento. |
| `resizable` | `yes/no/1/0` | Se puede redimensionar. |
| `width`, `height` | px | Tamaño de la ventana. |
| `outerWidth`, `outerHeight` | px | Tamaño **total** de la ventana. |
| `left`, `top` | px | Posición en pantalla. |

### Temporizadores

```
// Reloj: actualiza cada segundo
setInterval(function () {
  document.getElementById("reloj").innerHTML = new Date().toLocaleTimeString();
}, 1000);

// Una sola vez
setTimeout(function () {
  alert("Pasaron 2 segundos");
}, 2000);
```

> `setInterval` devuelve un **id** que se puede cancelar con `clearInterval(id)`.

---

## 24. Acceso a los nodos

| Método | Devuelve | Ejemplo |
|---|---|---|
| `getElementById(id)` | **Un** elemento (el `id` es único). **El más usado.** | `document.getElementById("encabezado")` |
| `getElementsByTagName(etiqueta)` | **Colección** de elementos con esa etiqueta. | `document.getElementsByTagName("p")` |
| `getElementsByName(name)` | Colección con ese atributo `name`. | `document.getElementsByName("teoria")` |
| `getElementsByClassName(clase)` | Colección con esa clase. | `document.getElementsByClassName("logo")` |
| `querySelector(selector)` | **El primer** elemento que coincide con el selector CSS. | `document.querySelector("p")` |
| `querySelectorAll(selector)` | **Todos** los que coinciden (colección). | `document.querySelectorAll("p.intro")` |

### `getElementsByTagName`

```
var parrafos = document.getElementsByTagName("p");
var primerParrafo = parrafos[0];

for (var i = 0; i < parrafos.length; i++) {
  var parrafo = parrafos[i];
}
```

```
// También se puede buscar dentro de un elemento
var enlaces = primerParrafo.getElementsByTagName("a");
```

`getElementsByName`

```
var parrafoEspecial = document.getElementsByName("teoria");
```

```
<p name="Evaluación">...</p>
<p name="teoria">...</p>
<p>...</p>
```

> Normalmente `name` es único, así que sirve para acceder directo al nodo. En **radiobuttons**, todos los relacionados comparten `name`, por lo que devuelve una **colección**.

`getElementById`

```
var cabecera = document.getElementById("encabezado");
```

```
<div id="encabezado">
  <a href="/" id="logo">...</a>
</div>
```

> Como el `id` es **único**, devuelve solo el nodo deseado.

`querySelector` / `querySelectorAll`

```
document.querySelector("p");
document.querySelector(".clase");
document.querySelector("#papa");
```

```
<h3>Add a background color to the first p element:</h3>
<p>This is a p element.</p>
<p>This is a p element.</p>

<script>
  document.querySelector("p").style.backgroundColor = "red";
</script>
```

```
<p class="intro">Hola Mundo.</p>
<p class="intro">Este es un ejemplo</p>
<p id="demo"></p>

<script>
  const x = document.querySelectorAll("p.intro");
  document.getElementById("demo").innerHTML =
    'El primer parrafo (index 0) con class="intro": ' + x[0].innerHTML;
</script>
```

> `querySelectorAll` acepta **cualquier selector CSS**: id, clase, tipo, atributos, valores de atributos.

---

## 25. Crear y eliminar nodos

### Crear elementos

```
// 1. Crear nodo de tipo Element
var parrafo = document.createElement("p");

// 2. Crear nodo de tipo Text
var contenido = document.createTextNode("Hola Mundo!");

// 3. Añadir el texto como hijo del párrafo
parrafo.appendChild(contenido);

// 4. Añadir el párrafo como hijo del body
document.body.appendChild(parrafo);
```

| Método | Qué crea |
|---|---|
| `createElement(etiqueta)` | Nodo **Element** ( `<p>` , `<div>` ...). |
| `createTextNode(texto)` | Nodo **Text** (contenido). |

| Método | Qué crea |
|---|---|
| `appendChild(hijo)` | Agrega el nodo como **último hijo**. |

### Eliminar nodos

`removeChild()` requiere el nodo a eliminar y **debe invocarse desde el padre**. La forma más segura de llegar al padre es `parentNode`.

```
var parrafo = document.getElementById("sacame");
parrafo.parentNode.removeChild(parrafo);
```

```
<p id="sacame">...</p>
```

---

## 26. Atributos, `innerHTML` y `style`

### `getAttribute` / `setAttribute`

```
<img src="stratocaster.jpg" alt="Guitarra Electrica" id="fender">
```

```
var imagenGrande = document.getElementById("fender");

alert(imagenGrande.getAttribute("src"));      // "stratocaster.jpg"

imagenGrande.setAttribute("src", "GuitarraElectrica.jpg");
```

| Método | Para qué |
|---|---|
| `getAttribute(nombre)` | Obtiene el valor de un atributo. |
| `setAttribute(nombre, valor)` | Fija el valor de un atributo. |

### `innerHTML`

- Es una de las formas que tiene JavaScript para **mostrar información**; se usa en librerías como **jQuery**.
- Sirve para **acceder y cambiar el texto HTML dentro de un elemento**.

```
<!DOCTYPE html>
<html>
<body>
  <h1>Mi Pagina</h1>
  <p>Esto es un parrafo</p>
  <p id="demo"></p>

  <script>
    document.getElementById("demo").innerHTML = 5 + 6;
  </script>
</body>
</html>
```

> `document.write` **borra** el contenido de la página; `innerHTML` escribe **dentro** de un elemento.

### `style`

Permite **acceder, cambiar y remover CSS** dentro de un elemento.

```
document.getElementById("intro").style.color = "#fff";
document.getElementById("intro").style.backgroundColor = "#f58220";

var brandColor = document.getElementById("intro").style.backgroundColor;
```

> Las propiedades CSS con guion pasan a **camelCase**: `background-color` → `backgroundColor`, `font-size` → `fontSize`.

---

## 27. `addEventListener`

Permite **agregar eventos a elementos HTML**. Sintaxis general:

```
elemento.addEventListener(evento, funcion);
```

```
<button id="myBtn">Apretar</button>
<p id="demo"></p>
```

```
<script>
  document.getElementById("myBtn").addEventListener("click", displayDate);

  function displayDate() {
    document.getElementById("demo").innerHTML = Date();
  }
</script>
```

```
<button id="myboton">Try it</button>
<p id="demo"></p>

<script>
  const element = document.getElementById("myboton");
  element.addEventListener("click", myFunction);

  function myFunction() {
    document.getElementById("demo").innerHTML = "Hola mundo";
  }
</script>
```

| Ventaja | Detalle |
| --- | --- |
| Separa HTML de JS | No se usan atributos `onclick` en el HTML. |
| Varios listeners | Se pueden registrar **varios** manejadores para el mismo evento. |
| Remoción | Se quita con `removeEventListener(evento, funcion)`. |

## 28. Errores típicos y checklist

| Error | Causa | Solución |
| --- | --- | --- |
| `X is not defined` | La variable no existe o está mal escrita. | Revisar mayúsculas y el ámbito. |
| `Cannot read properties of null` | El elemento no existe **todavía** al ejecutar el script. | Poner el script al final del `body` o esperar `DOMContentLoaded`. |
| `let` redeclarado da error | `let x` dos veces en el mismo ámbito. | Usar `var` o reasignar sin `let`. |
| `const` da error al cambiar | Se intenta redefinir una constante. | Usar `let` si el valor cambia. |
| `"2" + 5` da `"25"` | Coerción de tipos (concatenación). | Convertir con `parseInt()` / `Number()`. |
| `prompt` suma mal | `prompt` devuelve **string**. | `parseInt(prompt(...))`. |
| `is not a function` | Se llamó a una propiedad que no es función o el objeto no cargó. | Verificar el nombre del método. |
| Cambia la imagen equivocada | El `name` / `id` no coincide. | Revisar el atributo y la ruta. |
| La validación no frena el envío | Falta `return` en el `onsubmit`. | `onsubmit="return validar()"`. |
| `onclick` no hace nada | Error de sintaxis dentro del atributo o función no definida. | Revisar la consola del navegador. |
| `document.write` borra la página | Se ejecutó después de cargar el documento. | Usar `innerHTML`. |
| `new Array(3)` no tiene 3 elementos | `new Array(n)` fija el **largo**, no los valores. | Asignar cada posición o usar `[a, b, c]`. |
| `getElementsByClassName` no existe | Nombre mal escrito (`getElementByClassName`). | Es **Elements** en plural. |
| Fecha con mes incorrecto | `getMonth()` devuelve 0–11. | Sumar 1. |

### Checklist antes de entregar

- [ ] Scripts **al final del** `body` o dentro de `DOMContentLoaded`.
- [ ] Variables declaradas con `let` / `const` (o `var` si el navegador es viejo).
- [ ] Nombres coherentes y sin palabras reservadas.
- [ ] Conversión explícita de datos que vienen de `prompt` o de formularios.
- [ ] Comparaciones con `===` cuando importa el tipo.
- [ ] Bucles con condición de corte clara (evitar bucles infinitos).
- [ ] Eventos con `return false` cuando deben frenar la acción por defecto.
- [ ] Probado en la **consola del navegador** sin errores.

- [ ] Sin `console.log` de depuración en la entrega.
- [ ] Código indentado y comentado en las partes no obvias.
- [ ] Funciona con el documento ya cargado (DOM disponible).

---

## 29. Plantilla completa

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>JavaScript — Plantilla</title>
</head>
<body>

  <h1>Ejemplo JavaScript</h1>

  <p id="saludo"></p>
  <input type="text" id="nombre" placeholder="Tu nombre">
  <button id="btnSaludar">Saludar</button>

  <ul id="lista"></ul>

  <script>
    // 1. Datos y funciones
    const nombres = ["Ana", "Beto", "Carla"];

    function saludar(nombre) {
      return `Hola, ${nombre}!`;
    }

    function pintarLista() {
      const lista = document.getElementById("lista");
      lista.innerHTML = "";

      for (let i = 0; i < nombres.length; i++) {
        const item = document.createElement("li");
        item.textContent = nombres[i];
        lista.appendChild(item);
      }
    }

    // 2. Eventos cuando el DOM ya existe
    document.getElementById("btnSaludar").addEventListener("click", function () {
      const nombre = document.getElementById("nombre").value.trim();

      if (nombre === "") {
        alert("Escribí un nombre");
        return;
      }

      document.getElementById("saludo").innerHTML = saludar(nombre);
      nombres.push(nombre);
      pintarLista();
    });

    // 3. Inicialización
    pintarLista();
  </script>

</body>
</html>
```

### Versión mínima "de cátedra" (script en el body)

```
<html>
<head>
  <title>Mi página</title>
</head>
<body>
  <h2>Datos</h2>
  <div id="salida"></div>

  <script language="JavaScript">
    var nombre = prompt("¿Su nombre?", "");
    document.getElementById("salida").innerHTML =
```

```
      "<p>Hola " + nombre + "</p>";

    var hoy = new Date();
    document.write("<p>Hoy es " + hoy + "</p>");
  </script>
</body>
</html>
```

## 30. Ejercicios de repaso

1. **Salida:** mostrar por pantalla los números del 1 al 100 de 10 en 10 con `for` y con `while`.
2. **Coerción:** predecir el resultado de `"2" + 5`, `"2" * 5`, `"2" - 5 + "2"` y `2 * 3 + "ra"`.
3. **Variables:** explicar la diferencia entre `var`, `let` y `const` y qué pasa al redeclarar cada una.
4. **Funciones:** escribir `multiplicar(a, b)` que retorne el producto y usarla con `document.write`.
5. **Objetos:** crear un objeto `auto` con `marca`, `modelo` y un método `descripcion()` que use `this`.
6. **Arreglos:** cargar 5 frutas en un arreglo, ordenarlas con `sort()` y mostrarlas con `for...in`.
7. **Eventos:** validar con `onsubmit` que un campo no esté vacío y frenar el envío con `return false`.
8. **DOM:** crear un `<p>` con `createElement` + `createTextNode`, agregarlo con `appendChild` y luego eliminarlo con `removeChild`.
9. **String:** contar cuántas veces aparece la letra "a" en una frase usando `indexOf` o `split`.
10. **Template literal:** armar `"Total: $12.50"` a partir de `price` y `VAT` con `${}` y `toFixed(2)`.
11. **`querySelectorAll`:** pintar de rojo todos los párrafos con `class="intro"`.
12. **Temporizador:** mostrar un reloj que se actualice cada segundo con `setInterval`.

---

**Fin del cheatsheet.**
Si tenés que priorizar para el parcial: qué es JavaScript y sus características, diferencias con Java, las 4 formas de declarar variables (`var` / `let` / `const`), tipos y coerciones, operadores, control de flujo (`if`, `for`, `for...in`, `while`), funciones, objetos con `this`, `Math` / `Date`, **eventos** (niveles, `onClick` / `onChange` / `onSubmit`, `this`, `return false`), **strings** y template literals, y **DOM** (`getElementById`, `getElementsByTagName`, `querySelector(All)`, `createElement` / `appendChild` / `removeChild`, `getAttribute` / `setAttribute`, `innerHTML`, `style`, `addEventListener`).